```
    x**2
  end
end
```

However, there's another convention, associated with Jim Weirich (the creator of Rake), where braces are used for multiline blocks if the return value of the method is immediately used in a method chain like this:

```
def another_sample
  [1, 2, 3].map { |x|
    do_something_with(x)
  }.sort.first
end
```

In this example, the result of the map call is passed directly to sort, so the Weirich convention uses braces because the feeling is that end.sort.first looks odd.

Typically we put blank lines between method and class definitions, and only inside methods rarely. Ruby methods tend to be short, and needing internal space might be a sign that the method should be split.

Parentheses, especially for method calls, also have a couple of popular styles. The most common style is to use parenthesis for all method calls unless the method call is:

- A Kernel method like puts or p that is meant to look like a command.
- A class method being invoked as part of the definition of a class, like include or private.
- Used in a Ruby domain-specific language that doesn't use parentheses, like RSpec's it or describe.

No matter what, method calls with no arguments in Ruby should not get empty parentheses (except in the one case where super() has a different meaning than super).

You may sometimes see references to "Seattle Style" Ruby code, in which parentheses are only included if they are needed to resolve ambiguity.

Naming and Calling

Local variables, symbols, method names, and file names in Ruby use underscores to separate words, as in a_multi_word_name. Modules and classes that start with capital letters use intercaps to separate words, as in AMultiWordName. Our preference is to lowercase acronyms in intercapped names—as in HttpReceiver, not HTTPReceiver—on the grounds that it's easier to separate the words, but this is an example of where people's opinion definitely varies.

Constants are typically written in all caps and use underscores to separate names, as in A_MULTI_WORD_NAME.

Ruby tends not to prefix getters, setters, and predicate methods. *Getters* are typically the name of the attribute, *setters* are the name of the attribute with an = suffix, and *predicate methods* don't typically start with is or a similar prefix. Instead, predicate methods in Ruby end with ?.

The ! suffix on a method name indicates that the method is "dangerous," which in Ruby-speak means it either

- modifies the receiver of the message, or
- raises an exception or error.